DTU

Danmarks Tekniske Universitet

Written exam 15 December, 2021

Course title: Network Optimization **Course no:** 42115

Aids allowed: All

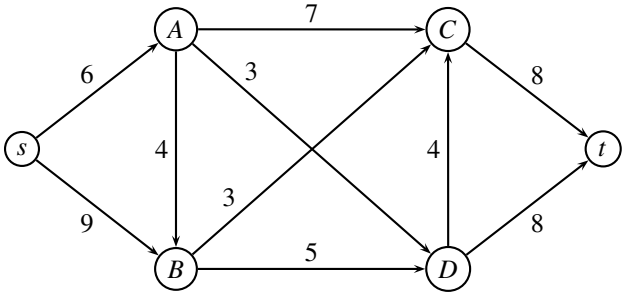
Exam duration: 4 hours

Weighting: The points are indicated at each question

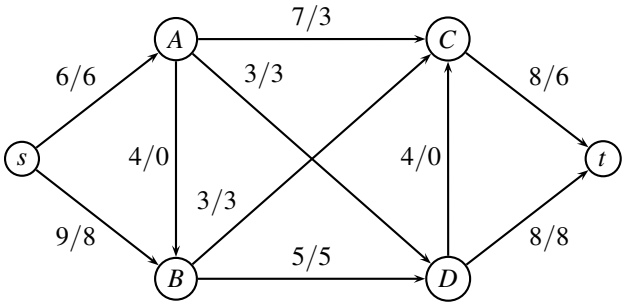
Language: You can write in Danish or English

Maximum flow problem

In the following network the edge weights denote the capacity. The problem is to find the maximum flow from s to t.

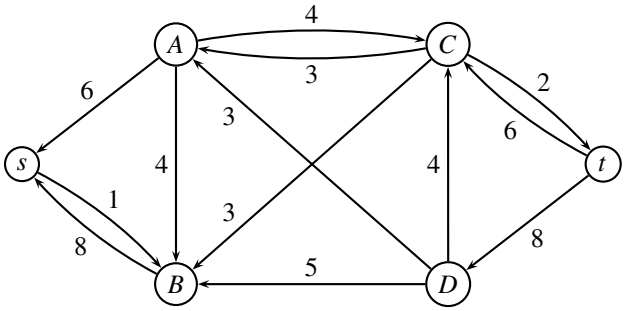


Answer 1 We use the paths *sACt* with capacity 6, *sBDt* with capacity 5, and *sBCADt* with capacity 3. This gives the flow



The flow is $6+5+3=14$. ■

Answer 2 The residual graph is



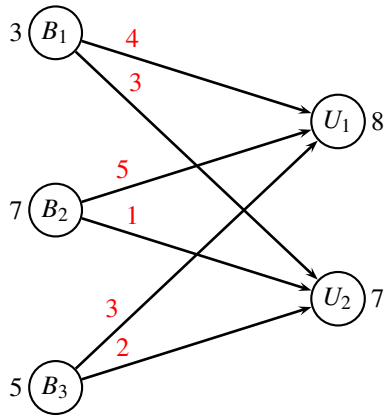
The cut is found by marking all nodes that are reachable from s. This is the set $S = \{s, B\}$. The corresponding cut has capacity $6+3+5 = 14$ The maximum flow is also $6+8 = 14$. ■

Transportation problem

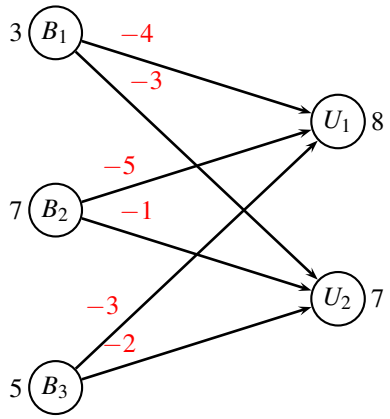
Three bakeries B_1, B_2, B_3 are delivering cakes to two universities U_1, and U_2. The bakeries are producing 3, 7 and 5 cakes respectively. University U_1 needs 8 cakes, while university U_2 needs 7 cakes. The profit of the cakes depends on the combination of bakery and university as given in the following matrix:

	B_1	B_2	B_3	demand
U_1	4	5	3	8
U_2	3	1	2	7
production	3	7	5	

We can formulate the problem as a transportation problem in maximization form:



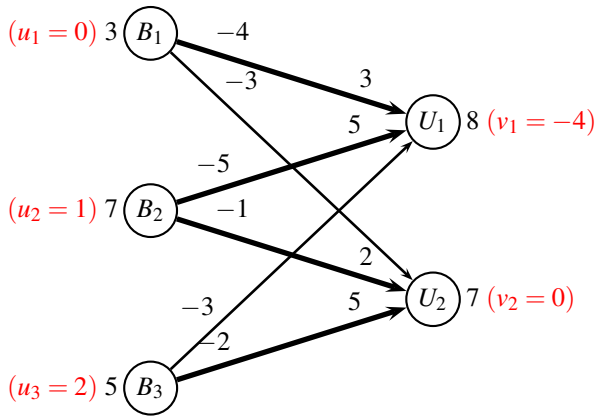
Answer 3 We can just multiply all costs with -1 since maximizing z is equivalent to minimizing $-z$.



Note, that the network simplex algorithm also works with negative costs.

Alternative solution: We can also first change sign, and then add a constant to all costs, to make them positive. Adding a constant will not affect the optimal solution vector (however, the constant should be subtracted from the objective value at the end of the solution algorithm). ■

Answer 4 We find the following greedy solution



The cost of the solution is $(-4)\cdot 3+(-5)\cdot 5+(-1)\cdot 2+(-2)\cdot 5=-49$.

Bold edges on the graph indicate the basis. Dual values (indicated with red) are found by finding the shortst path from node B_1, using only the bold edges.

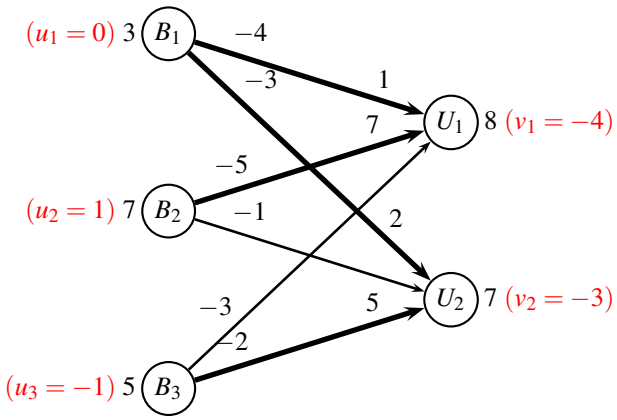
■

Answer 5 We now find the reduced costs as $\overline{c}_{ij} = c_{ij}+u_i-v_j$, getting

$$\overline{c}_{12} = -3+0-0=-3$$
$$\overline{c}_{31} = -3+2-(-4)=3$$

We choose edge (B_1,U_2) corresponding to the cycle (B_1,U_2,B_2,U_1). We can send at most $\delta = \min\{2,3\} = 2$ units along this cycle.

After the pivot operation we get the solution



The cost of the solution is $(-4)\cdot 1+(-3)\cdot 2+(-5)\cdot 7+(-2)\cdot 5=-55$.

The reduced costs are

$$\overline{c}_{22} = -1+1-(-3)=3$$
$$\overline{c}_{31} = -3+(-1)-(-4)=0$$

No negative reduced costs, so we terminate. The optimal solution value is $z_{\min} = -55$. ■

Answer 6 The flow on the edges is the same. The objective is $z_{\max} = -z_{min} = 55$. ■

Checking solution in CPLEX

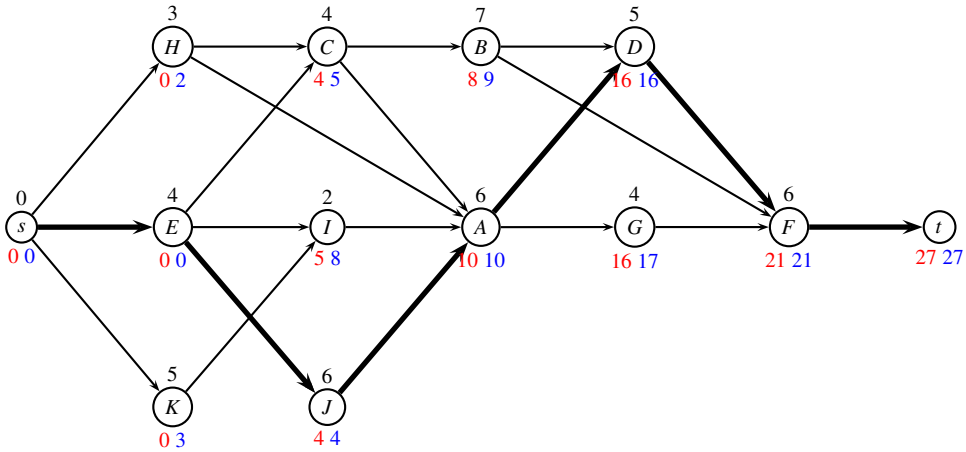
```
maximize
 4xB1U1+3xB1U2
+5xB2U1+1xB2U2
+3xB3U1+2xB3U2
subject to
 B1: xB1U1 + xB1U2 = 3
 B2: xB2U1 + xB2U2 = 7
 B3: xB3U1 + xB3U2 = 5
 U1: xB1U1 + xB2U1 + xB3U1 = 8
 U2: xB1U2 + xB2U2 + xB3U2 = 7
end

CPLEX> read transport
Problem 'transport.lp' read.
Read time = 0.00 sec. (0.00 ticks)
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 2 rows and 3 columns.
Aggregator did 3 substitutions.
All rows and columns eliminated.
Presolve time = 0.00 sec. (0.00 ticks)

Dual simplex - Optimal:  Objective =  5.5000000000e+01
Solution time =    0.00 sec.  Iterations = 0 (0)
Deterministic time = 0.01 ticks  (5.07 ticks/sec)

CPLEX> dis sol var -
Variable Name           Solution Value
xB1U2                         3.000000
xB2U1                         7.000000
xB3U1                         1.000000
xB3U2                         4.000000
All other variables in the range 1-6 are 0.
```

Critical path

Consider a critical path problem, in which each task i has a duration d_i, and a list of predecessors P_i. The predecessors P_i need to finish before task i can begin.

task i	duration d_i	predecessors P_i
A	6	C,H,I,J
B	7	C
C	4	E,H
D	5	A,B
E	4	
F	6	B,D,G
G	4	A
H	3	
I	2	E,K
J	6	E
K	5	

Answer 7 A topological ordering of the activities must ensure that all edges are pointing forward. This is the case in the following graph:



■

Answer 8 Now, we solve the problem

- *Forward procedure:* Starting at time zero ($U_s = 0$), calculate the earliest time U_j each activity j can be started $U_j = \max_{(i,j)\in E}(U_i + d_i)$. (marked with red)
- *Makespan:* $T = U_t = 27$.
- *Backward procedure:* Starting at time equal to the makespan ($V_t = T$), calculate the latest time V_j each activity j can be started so that this makespan is realized. $V_i = \min_{(i,j)\in E}(V_j - d_i)$ (marked with blue)

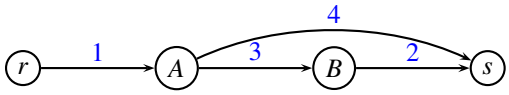
■

Answer 9 The critical path is found where $U_i = V_j$. The bold edges are marked on the above figure.
■

Max flow and min cut

Answer 10 We have the following two statements

a) **NO**. The following example shows a case where there are two different optimal solutions: Either 1 unit *rABs* or 1 unit *rAs*.



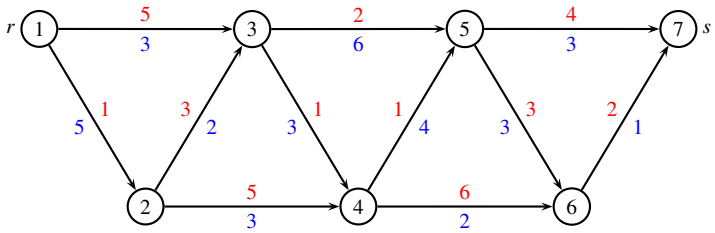
b) **YES**. If $(R,\overline{R})$ is the smallest among all cuts, then it will also be the smallest among all cuts when multiplying all edge weights by a positive constant.

Alternative solution: Let P be the original problem and P' the problem where all capacities are multiplied by $k > 0$. If (x_{ij}) is an optimal solution to P then the flow on all edges $(i,j) \in R \times \overline{R}$ is equal to the capacity u_{ij}. Now, $(k \cdot x_{ij})$ is also an optimal solution to P' and the flow on all edges $(i,j) \in R \times \overline{R}$ is equal to the capacity $k \cdot u_{ij}$. Hence, $(R,\overline{R})$ is a minimum cut.

■

Resource constrained shortest path (bidirectional method)

Consider the following resource constrained shortest path problem, where costs are red, and times are blue.



We want to find the shortest (cheapest) path from node $r = 1$ to node $s = 7$, with time constraint $T = 14$.

We first run forward dynamic programming where we stop extending states when their resource is above $T/2$.

node 1		node 2		node 3		node 4		node 5		node 6		node 7	
cost	time	cost	time	cost	time	cost	time	cost	time	cost	time	cost	time
0	0	1	5	4	7	5	10	6	13	12	8		
				5	3	6	6	7	9				

We then run backward dynamic programming where we also stop extending states when their resource is above $T/2$.

node 1		node 2		node 3		node 4		node 5		node 6		node 7	
cost	time	cost	time	cost	time	cost	time	cost	time	cost	time	cost	time
14	9	10	10	6	9	5	7	4	3	2	1	0	0
		12	8	9	6	8	3						
		13	6										

Answer 11 We merge forward and backward states in all nodes, deleting states that exceed time constraint T. Infeasible states are marked with brackets. Dominated (but feasible) states are marked with paranthesis.

node 1	
cost	time
14	9

node 2	
cost	time
[11	15]
13	13
14	11

node 3	
cost	time
[10	16]
11	12
(13	13)
14	9

node 4	
cost	time
[10	17]
11	13
(13	13)
14	9

node 5	
cost	time
[10	16]
11	12

node 6	
cost	time
14	9

node 7	
cost	time

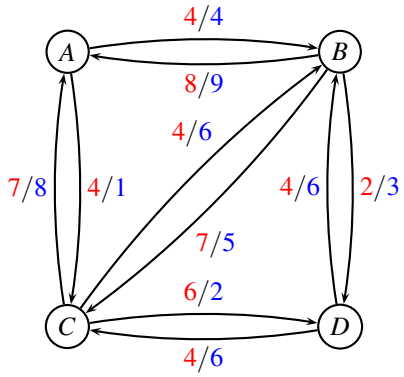
■

Answer 12 The shortest (cheapest) path is found as: (cost,time) $= (11, 12)$. ■

Repositioning of empty containers

We are considering the liner shipping network ASIANET from the project assignment. The network, costs, and capacities are **unchanged**.

ASIANET



For edge (i, j) the first number (red) is the cost c_{ij} and the second number (blue) is the capacity u_{ij}.

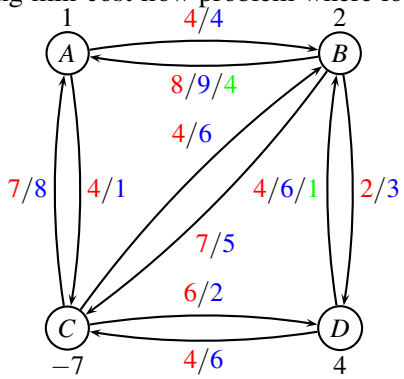
Due to the imbalance in trade, we need to reposition empty constainers. For ASIANET, we have the following demand for containers:

node	A	B	C	D
demand	1	2	-7	4

(*)

Negative demands denote a surplus of containers. It is assumed that all empty containers are equal. Furthermore, we have some **new lower bounds** on the edges: Edge (B,A) has a lower bound of $\ell_{BA} = 4$, and ege (D,B) has a lower bound of $\ell_{DB} = 1$.

Answer 13 We have the following min-cost flow problem where lower bounds are green.



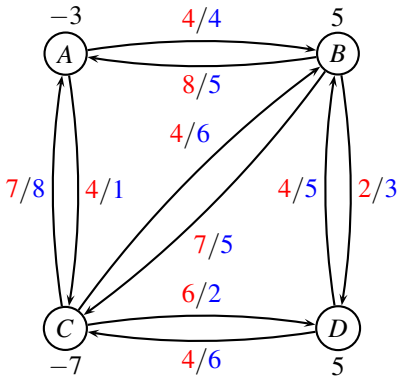
We use the following transformation to handle lower bounds: Assume that edge (i, j) has upper bound u_{ij} and lower bound ℓ_{ij}. Replace the upper bound with $u_{ij} := u_{ij} - \ell_{ij}$. Set the demand at node i to $b_i := b_i + \ell_{ij}$. Set the demand at node j to $b_j := b_j - \ell_{ij}$. In our case, we get

$$u_{BA} := u_{BA} - \ell_{BA} = 9 - 4 = 5, \qquad b_B := b_B + \ell_{BA} = 2 + 4 = 6, \qquad b_A := b_A - \ell_{BA} = 1 - 4 = -3$$

next we have

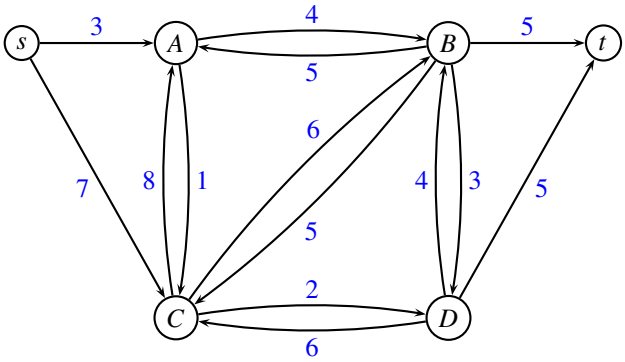
$$u_{DB} := u_{DB} - \ell_{DB} = 6 - 1 = 5, \qquad b_D := b_D + \ell_{DB} = 4 + 1 = 5, \qquad b_B := b_B - \ell_{DB} = 6 - 1 = 5$$

In this way we get the ordinary min-cost flow problem:

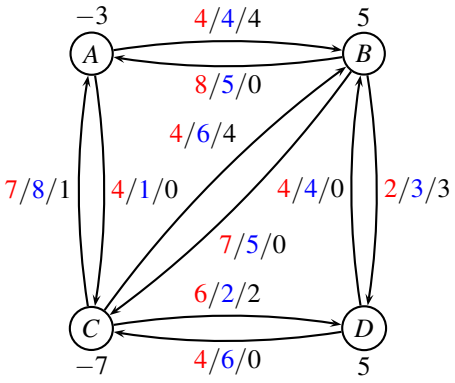


After having solved the problem, we need to add ℓ_{ij} to the flow variables to find the optimal solution. ■

Answer 14 We solve our problem using the *augmenting cycle algorithm*. First, an initial solution is found by adding a source s and terminal t. The source node s is connected to all nodes i having demand $b_i < 0$ with an edge having capacity $-b_i$. All nodes i having demand $b_i > 0$ are connected to the terminal node t with an edge having capacity b_i. Edge costs are ignored.

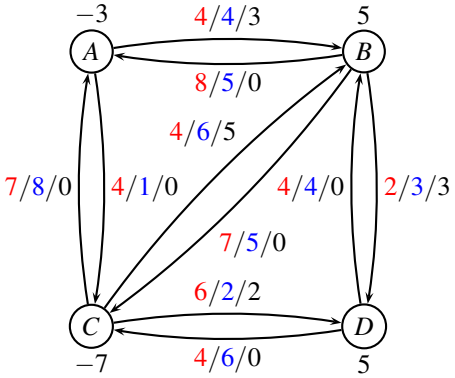


We now solve a maximum-flow problem using the augmented path algorithm. First, we find path *sCABt* with capacity 4. Next, we find path *sCBDt* with capacity 3. Then, we find path *sACDt* with capacity 2. Finally, we find path *sACBt* with capacity 1. This gives the initial solution to the minimum cost flow problem (black numbers indicate the flow):



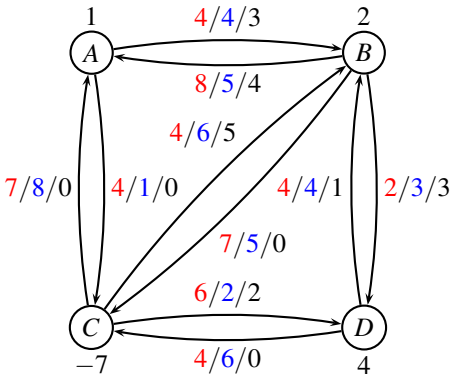
The cost is $z = 4 \cdot 4 + 7 \cdot 1 + 4 \cdot 4 + 2 \cdot 3 + 6 \cdot 2 = 57$.

An augmenting cycle is $CBAC$ with cost $4 - 4 - 7 = -7$ and capacity $\min\{2,4,1\} = 1$. We flow 1 unit along the cycle getting:



No more augmenting cycles can be found, so the algorithm terminates. The cost is $z = 4 \cdot 3 + 4 \cdot 5 + 2 \cdot 3 + 6 \cdot 2 = 50$. ■

Answer 15 To find the optimal solution to the original problem (*) we add lower bounds $\ell_{BA} = 4$ and $\ell_{DB} = 1$, getting:



The final cost is $z = 4 \cdot 3 + 4 \cdot 5 + 2 \cdot 2 + 6 \cdot 2 + 8 \cdot 4 + 4 \cdot 1 = 86$. ■

Checking solution in CPLEX

```
minimize
 4xAB+4xAC+8xBA+7xBC+2xBD+7xCA+4xCB+6xCD+4xDB+4xDC
subject to
 A: -xAB-xAC+xBA+xCA = 1
```

```
 B: xAB-xBA-xBC-xBD+xCB+xDB = 2
 C: xAC+xBC-xCA-xCB-xCD+xDC = -7
 D: xBD+xCD-xDB-xDC = 4
 UAB: xAB <= 4
 UAC: xAC <= 1
 UBA: xBA <= 9
 LAB: xBA >= 4
 UBC: xBC <= 5
 UBD: xBD <= 3
 UCA: xCA <= 8
 UCB: xCB <= 6
 UCD: xCD <= 2
 UDB: xDB <= 6
 LDB: xDB >= 1
 UDC: xDC <= 6
end

CPLEX> opt
Dual simplex - Optimal:  Objective =  8.6000000000e+01
Solution time =    0.04 sec.  Iterations = 0 (0)
Deterministic time = 0.01 ticks  (0.37 ticks/sec)

CPLEX> dis sol var -
Variable Name           Solution Value
xAB                          3.000000
xBA                          4.000000
xBD                          3.000000
xCB                          5.000000
xCD                          2.000000
xDB                          1.000000
All other variables in the range 1-10 are 0.
```

Multicommodity-flow

We consider again the route net ASIANET where we have **new demands** as follows:

k	s	t	d
1	C	B	4
2	A	D	2
3	D	A	6

Thus, commodity $k = 1$ is shipping four containers from C to B, commodity $k = 2$ is shipping two containers from A to D, and commodity $k = 3$ is shipping six containers from D to A.

Answer 16 We have the routes r outlined in the below table. The cost of sending d_k units along a route r is calculated as $d_k \sum_{(i,j)\in r} c_{ij}$.

number	commodity k	route r	d_k	$\sum c_{ij}$	cost
1	1	CAB	4	11	44
2	1	CB	4	4	16
3	2	ABCD	2	17	34
4	2	ABD	2	6	12
5	3	DBA	6	12	72
6	3	DCA	6	11	66
7	3	DCBA	6	16	96

This gives the path formulation:

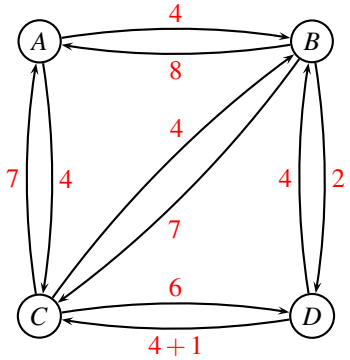
```
minimize
+44x1+16x2+34x3+12x4+72x5+66x6+96x7
subject to
UAB: +4x1    +2x3+2x4              <= 4
UAC:                               <= 1
UBA:                 +6x5    +6x7 <= 9
UBC:          +2x3                 <= 5
UBD:              +2x4             <= 3
UCA: +4x1                 +6x6     <= 8
UCB:     +4x2                 +6x7 <= 6
UCD:          +2x3                 <= 2
UDB:                 +6x5          <= 6
UDC:                      +6x6+6x7 <= 6

K1: +x1+x2 = 1
K2: +x3+x4 = 1
K3: +x5+x6+x7 = 1
end
```

■

We solve the problem through column generation. After a few iterations we have the dual variables: $y_{DC} = -1$ (all other $y_{ij} = 0$), while $\overline{y}_1 = 16$, $\overline{y}_2 = 34$, $\overline{y}_3 = 72$.

Here, y_{ij} denote the dual variable corresponding to capacity constraints, and $\overline{y}_k$ denote the dual variable corresponding to commodity constraints.

Answer 17 We update the edge costs for edge (D,C) getting the graph:



We find

$k = 1$: (demand $d_1 = 4$). We remove edges AC, BD, CD (due to capacity). Shortest path CB (route 1) with cost $z = 4$. Reduced cost is $d_1 \cdot z - \overline{y}_1 = 4 \cdot 4 - 16 = 0$.

$k = 2$: (demand $d_2 = 2$) We remove edge AC (due to capacity). Shortest path ABD (route 4) with cost $z = 6$. Reduced cost is $d_2 \cdot z - \overline{y}_2 = 2 \cdot 6 - 34 = -22$.

$k = 3$: (demand $d_3 = 6$) We remove edges AB, AC, BC, BD and CD (due to capacity). Shortest path DCA (route 6) with cost $z = 12$. Reduced cost is $d_3 \cdot z - \overline{y}_3 = 6 \cdot 12 - 72 = 0$. (**Alternative solution:** Shortest path DBA (route 5) with cost $z = 12$.)

The most negative reduced cost is obtained with path ABD (route 4) with reduced cost -22.

Checking solution in CPLEX

```
\ Example 4068b

minimize
+44x1+16x2+34x3+12x4+72x5+66x6+96x7
subject to
UAB: +4x1    +2x3+2x4             <= 4
UAC:                              <= 1
UBA:                 +6x5    +6x7 <= 9
UBC:         +2x3                 <= 5
UBD:             +2x4             <= 3
UCA: +4x1                +6x6     <= 8
UCB:     +4x2                +6x7 <= 6
UCD:         +2x3                 <= 2
UDB:                 +6x5         <= 6
UDC:                     +6x6+6x7 <= 6

K1: +x1+x2 = 1
K2: +x3+x4 = 1
K3: +x5+x6+x7 = 1

R1: x1 <= 0
\\ R2: x2 <= 0
\\ R3: x3 <= 0
R4: x4 <= 0
\\ R5: x5 <= 0
\\ R6: x6 <= 0
R7: x7 <= 0
end

CPLEX> read iter0
Problem 'iter0.lp' read.
Read time = 0.00 sec. (0.00 ticks)
CPLEX> opt
Tried aggregator 1 time.
```

```
LP Presolve eliminated 16 rows and 7 columns.
All rows and columns eliminated.
Presolve time = 0.00 sec. (0.00 ticks)

Dual simplex - Optimal:  Objective =  1.1600000000e+02
Solution time =    0.00 sec.  Iterations = 0 (0)
Deterministic time = 0.01 ticks  (9.26 ticks/sec)

CPLEX> dis sol var -
Variable Name           Solution Value
x2                            1.000000
x3                            1.000000
x6                            1.000000
All other variables in the range 1-7 are 0.
CPLEX> dis sol dua -
Constraint Name             Dual Price
UDC                          -1.000000
K1                           16.000000
K2                           34.000000
K3                           72.000000
R4                          -22.000000
All other dual prices in the range 1-16 are 0.
```

■

Research planning

A research company has to handle 3 projects, P_1, P_2, P_3, during the next 5 months.

project	start month	end month	man-months needed
P_1	1	4	14
P_2	1	5	16
P_3	3	5	8

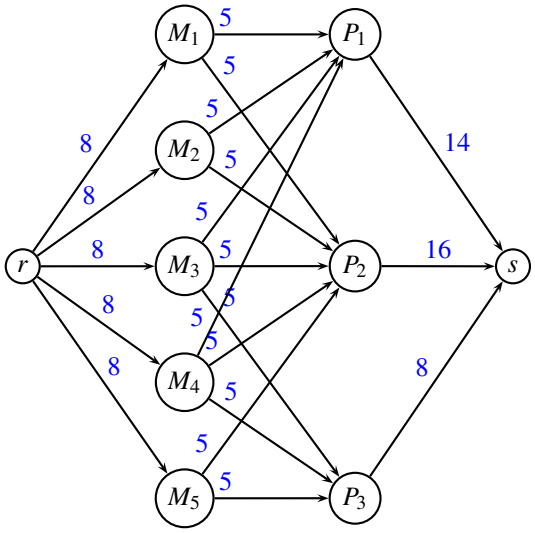
Project P_1 runs from month 1 to 4, P_2 runs from month 1 to 5, and P_3 runs from month 3 to 5. The projects require, respectivey, 14, 16, and 8 man-months. Each month, 8 researchers are available, however not more than 5 researchers can work on the same project each month.

Answer 18 We create a node for each month $M_1 \ldots, M_5$ and a node for each project $P_1, \ldots, P_3$. If project P_j is running in month M_i we connect the two nodes with an edge of capacity 5.

A root node r and sink node s is added. The root node r is connected to each month M_i with edges having capacity 8. All project nodes P_j are connected to the sink with an edge of capacity equal to the man-months needed on the project.

Now the problem is a maximum-flow problem. If we can find a solution that satisifies all demand for man-months (i.e. a flow of value $14+16+8$) then a feasible solution exists.

The flow on edges (M_i, P_j) denotes how many researchers should be assigned to project P_j in month M_i.



The above figure illustrates the formulation as a maximum flow problem. ■